

STQA - III

TY Bsc IT Mumbai University

Boundary Value Testing (BVT)

- **Boundary Value Testing (BVT)** is a **black-box testing technique** that focuses on testing the **edges (boundaries) of input domains** rather than the center. Since errors often occur at boundaries, this method helps in detecting edge-case failures in a system

Types of Boundary Value Testing

1. **Single Variable BVT** – Testing boundaries for one input parameter.
2. **Multiple Variable BVT** – When multiple inputs exist, combinations of boundary values are tested.
3. **Robust Boundary Testing** – Includes invalid boundary values to test error handling.

Key Concepts of Boundary Value Testing:

1. **Boundary Values** – The values that lie at the **extremes** (minimum and maximum) of an input range.
2. **Off-by-One Errors** – Issues that occur when conditions like `<`, `<=`, `>`, `>=` are incorrectly implemented.
3. **Testing Boundary Points** – Involves testing values:
 - **Minimum Boundary** – The lowest valid input.
 - **Just Below Minimum** – A value just below the valid range.
 - **Just Above Minimum** – A value slightly above the lowest valid input.
 - **Maximum Boundary** – The highest valid input.
 - **Just Below Maximum** – A value slightly below the upper limit.
 - **Just Above Maximum** – A value just beyond the valid range.

Example of Boundary Value Testing

Example 1: Valid Age Range (18–60)

- Lower Boundary Cases:
 - Just Below Minimum: 17 (Invalid)
 - Minimum Value: 18 (Valid)
 - Just Above Minimum: 19 (Valid)
- Upper Boundary Cases:
 - Just Below Maximum: 59 (Valid)
 - Maximum Value: 60 (Valid)
 - Just Above Maximum: 61 (Invalid)

Robust Boundary Value Testing

- **Robust Boundary Value Testing** is an extension of **Boundary Value Testing (BVT)** that includes **both valid and invalid boundary values** to check how the system handles edge cases and unexpected inputs.
- This technique ensures that the system can handle both **normal** and **out-of-range inputs** gracefully without crashing or producing incorrect results.

- **Key Concepts of Robust Boundary Value Testing**
- **Tests Standard Boundary Values** (Minimum, Maximum, Just Below, Just Above).
- **Includes Invalid Inputs** (Out-of-Bounds Cases) to check system robustness.
- **Prevents Crashes and Unexpected Behavior** by verifying proper error handling.

Example of Robust Boundary Value Testing

Example 1: Valid Age Range (18–60)

If an application accepts ages between 18 and 60, we define boundary values as:

Case	Test Value	Expected Outcome
Just Below Minimum	17	Invalid
Minimum Value	18	Valid
Just Above Minimum	19	Valid
Just Below Maximum	59	Valid
Maximum Value	60	Valid
Just Above Maximum	61	Invalid
Far Below Minimum	-5	Invalid
Far Above Maximum	100	Invalid

Comparison: Normal vs. Robust BVT

Boundary Value Testing	Robust Boundary Value Testing
Tests only valid boundary values	Tests both valid and invalid boundary values
Focuses on edge cases within limits	Checks system behavior outside limits
Ensures correct results within boundaries	Ensures proper error handling for out-of-range inputs

Advantages of Robust Boundary Value Testing

- ✔ Detects more **edge-case defects** than normal BVT
- ✔ Ensures **stability** by checking unexpected inputs
- ✔ Improves **error handling and validation**
- ✔ Prevents **security vulnerabilities** caused by unhandled inputs

Worst Case Boundary Value Testing

- **Worst Case Boundary Value Testing** is an extension of **Boundary Value Testing (BVT)** that tests **all possible combinations of extreme values** for multiple input variables.
- Unlike standard **BVT**, which tests boundaries for each variable independently, **Worst Case BVT** considers **combinations of boundary values for multiple inputs**, ensuring that the system performs well under extreme conditions.

Traditional Equivalence Class Testing

- **Equivalence Class Testing** is a black-box testing technique that divides input data into different groups or classes to reduce the number of test cases while ensuring adequate test coverage.
- **Traditional Equivalence Class Testing** follows a structured approach to identify valid and invalid input partitions based on system requirements

Key Concepts of Worst Case Boundary Value Testing:

1. **Considers Multiple Input Variables** – Tests boundary values for all inputs simultaneously.
2. **Tests All Combinations of Boundary Values** – Instead of testing individual boundaries, it combines different boundary cases for all inputs.
3. **Increases Test Coverage** – Identifies errors that may occur due to **interactions between input variables**.
4. **More Exhaustive Than Standard BVT** – Ensures robustness by testing extreme scenarios.

Example of Worst Case Boundary Value Testing

Example 1: Valid Age & Salary Range

Consider a system that accepts **age** (18–60) and **salary** (20,000–100,000). The boundary values are:

- **Age:** Min = 18, Just Above Min = 19, Just Below Max = 59, Max = 60
- **Salary:** Min = 20,000, Just Above Min = 21,000, Just Below Max = 99,000, Max = 100,000

Possible Worst-Case BVT Combinations:

Test Case	Age	Salary
1	18 (Min)	20,000 (Min)
2	18 (Min)	21,000 (Just Above Min)
3	18 (Min)	99,000 (Just Below Max)
4	18 (Min)	100,000 (Max)
5	19 (Just Above Min)	20,000 (Min)
6	19 (Just Above Min)	100,000 (Max)
7	59 (Just Below Max)	20,000 (Min)
8	59 (Just Below Max)	100,000 (Max)
9	60 (Max)	20,000 (Min)
10	60 (Max)	100,000 (Max)

Comparison: Normal vs. Worst Case BVT

Normal BVT	Worst Case BVT
Tests boundary values individually	Tests all combinations of boundary values
Fewer test cases	More exhaustive test cases
Suitable for simpler inputs	Ideal for complex, multi-input scenarios
Focuses on individual limits	Identifies interaction-related issues

Advantages of Worst Case BVT

- ✔ Detects hidden bugs due to variable interactions
- ✔ Improves system robustness in extreme conditions
- ✔ Higher defect detection rate compared to standard BVT
- ✔ Essential for mission-critical applications (e.g., banking, healthcare, aviation)

Special Value Boundary Testing

Special Boundary Value Testing (Special BVT) is an extension of Boundary Value Testing that focuses on testing values that are not just at the strict boundaries but are significant for the given application. These values might include:

- Default values (e.g., 0, NULL)
- Negative values (if applicable)
- Extreme but valid values
- Special characters or unique constraints

Key Concepts of Special BVT:

- 1. **Identifies critical special values** – Tests values beyond just minimum and maximum boundaries.
- 2. **Considers domain-specific edge cases** – Includes values that are significant for the specific application.
- 3. **Focuses on system behavior for unusual but valid inputs.**
- 4. **Covers unique cases like NULL, zero, and predefined limits.**

Comparison: Standard BVT vs. Special BVT

Standard BVT	Special BVT
Tests only boundary values	Tests special and extreme cases
Uses minimum, maximum, and adjacent values	Includes zero, NULL, special characters, negative values, etc.
Covers general boundary-related bugs	Detects application-specific defects

Example 1: Testing a Banking System (Withdrawal Limits: ₹500 – ₹50,000)

Test Case	Input Value	Reason
1	500	Lower boundary
2	499	Just below lower boundary (Invalid)
3	501	Just above lower boundary
4	50,000	Upper boundary
5	50,001	Just above upper boundary (Invalid)
6	0	Special case (Invalid for withdrawal)
7	-100	Special case (Negative withdrawal, Invalid)
8	25,000	Mid-range, valid case
9	999,999	Extreme but invalid input
10	NULL	Special case (Invalid input)

Random Testing

- **Random Testing** is a **black-box testing technique** where test cases are generated randomly without following predefined test design techniques. The goal is to check how the system behaves under different, sometimes unexpected, inputs.

Key Characteristics of Random Testing:

1. Inputs are selected randomly – No structured test design.
2. Does not require prior knowledge of the system's internal structure.
3. Automated tools are often used to generate large sets of test data.
4. Can uncover unexpected defects, especially in systems handling large or unstructured data.

Process of Random Testing:

1. **Define Input Range** – Identify valid and invalid inputs.
2. **Generate Random Test Cases** – Randomly select values within and outside the valid range.
3. **Execute Tests** – Run the system with the randomly generated inputs.
4. **Compare Results** – Verify outputs against expected behavior.
5. **Log Defects** – Identify unexpected crashes or failures.

Example of Random Testing

Example 1: Login System (Username & Password)

- Randomly generate **different username-password combinations**, including:
 - Valid and invalid usernames.
 - Randomly generated passwords (empty, special characters, long strings, etc.)
 - Extremely long or short inputs.

Key Concepts of Traditional Equivalence Class Testing

1. Partitioning Input Domain

- The input domain is divided into **equivalence classes** where all inputs in a class are expected to produce the same behavior.
- It helps in minimizing redundant test cases.

2. Types of Equivalence Classes

- **Valid Equivalence Class:** Contains input values that are within the acceptable range.
- **Invalid Equivalence Class:** Includes input values that are outside the valid range or boundary conditions.

3. Test Case Selection

- A test case is chosen from each equivalence class, assuming that if one test case from a class works correctly, all others in that class will too.
- Helps in reducing the overall number of test cases while maintaining test effectiveness.

Example of Traditional Equivalence Class Testing

Scenario: Validating Age Input (18-60 years)

Equivalence Class	Test Input	Expected Outcome
Valid Class	25 (within range)	Accepted
Invalid Class (Below range)	15 (below 18)	Rejected
Invalid Class (Above range)	65 (above 60)	Rejected

By testing only a few selected values from each class, we efficiently validate the system's response.

Advantages of Equivalence Class Testing

- Reduces the number of test cases while ensuring good test coverage.
- Saves time and effort by avoiding redundant testing.
- Helps in identifying defects early by testing valid and invalid inputs.

Improved Equivalence Class Testing

- **Improved Equivalence Class Testing** is an enhancement of **Traditional Equivalence Class Testing**, focusing on increasing test coverage and effectiveness by considering additional factors such as boundary values, multiple variable interactions, and error-prone cases.
- It refines the traditional approach by reducing the risk of missing critical defects.

Key Enhancements in Improved Equivalence Class Testing

1. Inclusion of Boundary Values

- Traditional equivalence class testing selects a random value from each class, but **improved** testing ensures that boundary values of each class are explicitly tested.
- Example: If the valid input range is **18–60**, instead of just selecting **25**, we also test **18 and 60** to ensure system stability at edges.

2. Handling Overlapping Equivalence Classes

- Some test cases may belong to multiple equivalence classes (e.g., an input value could be both a valid number and a special character).
- Improved testing considers such interactions to detect hidden defects.

3. Considering Multiple Input Variables

- Instead of testing single input fields independently, combinations of input variables are tested together to evaluate their dependencies.
- Example: In a loan application, both **age (18-60)** and **income (above ₹25,000)** must be valid for approval. Testing both variables in combination ensures completeness.

4. Negative and Invalid Class Refinements

- Traditional testing only considers a broad invalid class, but improved testing divides invalid inputs into **more granular subsets** such as:
 - Completely out of range (e.g., age = -5)
 - Just outside valid range (e.g., age = 17 or 61)
 - Invalid data type (e.g., age = "twenty-five")
 - Empty input cases (e.g., age = "")

5. Testing Special and Edge Cases

- In addition to normal test cases, improved equivalence class testing includes special cases like:
 - Null or empty values
 - Large or extreme values
 - Unexpected inputs (e.g., SQL injection, special characters)

Example: Improved Equivalence Class Testing for Age Validation (18-60 years)

Equivalence Class	Test Input	Expected Outcome
Valid Class	25	Accepted
Boundary Cases	18, 60	Accepted
Invalid (Below range)	15, 17	Rejected
Invalid (Above range)	61, 100	Rejected
Invalid (Data Type)	"twenty-five", -5, NULL	Rejected

Edge Testing

- **Edge Testing** is a software testing technique that focuses on evaluating system behavior at the extreme boundaries of input domains.
- It helps identify defects that occur at the **upper and lower limits** of valid and invalid inputs, ensuring system stability under edge conditions.

Key Concepts of Edge Testing

1. Testing at Extremes

- Instead of picking random values within the input range, **Edge Testing** specifically tests values **at or near the boundaries** of the system's constraints.
- Helps in detecting issues like **off-by-one errors**, **buffer overflows**, and **unexpected system crashes**.

2. Types of Edge Testing

- **Single Variable Edge Testing**: Testing boundary values of a single input field.
- **Multi-Variable Edge Testing**: Testing edge values of multiple inputs simultaneously.

3. Closely Related to Boundary Value Analysis (BVA)

- While **Equivalence Class Testing** picks **representative values**, **Boundary Value Analysis (BVA)** focuses on the actual edge values.
- **Edge Testing goes beyond BVA** by considering system-level behavior at extreme points.

Real-World Applications of Edge Testing

- ♦ **ATM Withdrawal Limit:** Testing withdrawal limits at ₹10,000 (max limit) and ₹10,001 (exceeding limit).
- ♦ **Password Length:** Testing passwords at 8 characters (minimum limit) and 7 characters (below limit).
- ♦ **E-commerce Cart:** Testing maximum allowed items in a shopping cart.
- ♦ **File Uploads:** Checking upload limits by testing just below, at, and above the file size restriction.

Decision Table Testing Technique

Decision Table Testing is a **black-box testing technique** used for testing complex business logic where multiple conditions influence the system's behavior. It provides a **systematic way** to identify all possible input combinations and their corresponding expected outputs.

1. Used for Complex Business Rules

- Helpful when multiple **conditions (inputs)** affect multiple **actions (outputs)**.
- Ensures that all possible combinations are tested.

2. Structure of a Decision Table

A **decision table** consists of:

- **Conditions (Inputs):** Different factors affecting the system.
- **Actions (Outputs):** Expected results based on conditions.
- **Rules:** Each combination of conditions leading to a particular action.

3. Types of Decision Tables

- **Limited Entry Decision Table:** Conditions have only **Yes/No (True/False)** values.
- **Extended Entry Decision Table:** Conditions can have multiple possible values.

Example of Decision Table: Login System

Scenario: A user can log in only if both **Username** and **Password** are correct.

Rule No.	Username Correct?	Password Correct?	Login Successful?
1	Yes	Yes	✓ Yes
2	Yes	No	✗ No
3	No	Yes	✗ No
4	No	No	✗ No

Real-World Applications of Decision Table Testing

- ✓ **Bank Loan Approval:** Checking loan eligibility based on income, credit score, and existing loans.
- ✓ **E-commerce Discounts:** Applying discounts based on cart value, promo codes, and user membership.
- ✓ **Insurance Claims:** Evaluating claims based on policy type, claim reason, and coverage limits.

Cause-and-Effect Graphing in Software Testing

Cause-and-Effect Graphing is a **black-box testing technique** used to derive test cases by analyzing the relationship between **inputs (causes)** and **outputs (effects)**. It helps identify test conditions systematically, especially when dealing with **complex business logic and dependencies**.

1. Identifies Relationships Between Inputs and Outputs

- **Causes:** Inputs or conditions that affect system behavior.
- **Effects:** Expected outcomes or system responses.

2. Graphical Representation

- The **cause-and-effect graph** visually represents dependencies between inputs and outputs.
- Logical relationships such as **AND, OR, NOT** are used to define rules.

3. Conversion to Decision Table

- The graph is converted into a **decision table**, making it easier to generate **test cases**.

Example: ATM Cash Withdrawal

Scenario: A user can successfully withdraw cash from an ATM if:

- The **PIN is correct** (Cause 1)
- The **account has sufficient balance** (Cause 2)
- The **ATM has cash available** (Cause 3)

Graph Representation:

- If all **three conditions** are met → Withdrawal **success** (Effect 1)
- If **any condition fails** → Withdrawal **fails** (Effect 2)

Decision Table Based on Cause-and-Effect Graph

Rule No.	Correct PIN	Sufficient Balance	ATM Has Cash	Withdrawal Success?
1	Yes	Yes	Yes	✓ Yes
2	No	Yes	Yes	✗ No
3	Yes	No	Yes	✗ No
4	Yes	Yes	No	✗ No

Path Testing in Software Testing

Path Testing is a **white-box testing technique** used to ensure that every possible execution path in a program is tested at least once. It helps identify logical errors, missing conditions, and unreachable code.

1. Based on Control Flow Graph (CFG)

- The program is represented as a **graph** where:
 - **Nodes** represent statements or decisions.
 - **Edges** represent control flow (execution paths).

2. Types of Paths Tested

- **Independent paths:** Unique execution flows through the code.
- **Decision paths:** Testing different conditions in decision-making statements (`if`, `switch`).
- **Loop paths:** Evaluating loops (`for`, `while`) for zero, one, and multiple iterations.

Data Flow Testing in define

- **Data Flow Testing** is a **structural testing technique** that focuses on the **flow of data** through a program to identify **anomalies, improper variable usage, and data dependency issues**.
- It examines the **lifecycle of variables**, from their **definition (creation) to their usage and eventual termination (deletion)**, to detect potential errors.

Key Aspects of Data Flow Testing:

1. **Variable Tracking** – Analyzes how variables are defined, used, and killed (discarded).
2. **Def-Use Pairs** – Identifies the relationship between where variables are **defined** (assigned a value) and **used** (read or modified).
3. **Types of Uses:**
 - **Computational Use (c-use)** – When a variable is used in a calculation.
 - **Predicate Use (p-use)** – When a variable is used in decision-making (e.g., in loops or conditionals).
4. **Anomaly Detection:** Identifies issues like:
 - Undefined variables being used.
 - Variables being used without prior assignment.
 - Variables being defined but never used.

Advantages of Data Flow Testing:

- Detects uninitialized variables and redundant assignments.
- Enhances code reliability by ensuring proper variable usage.
- Helps in finding dead code and potential runtime errors.

This technique is particularly useful in white-box testing to ensure that a program correctly manages data flow and prevents logical errors.

Slice-Based Testing

- **Slice-Based Testing** is a **white-box testing technique** that focuses on **analyzing specific portions (slices) of a program** related to a particular computation, variable, or function.
- It helps in understanding program behavior by isolating relevant code segments and examining their correctness.

Key Concepts of Slice-Based Testing:

1. **Program Slicing** – The process of extracting parts of a program that influence or are influenced by a particular variable or computation.
2. **Types of Program Slices:**
 - **Static Slice** – Extracts code that **may** affect the value of a variable, considering all possible program executions.
 - **Dynamic Slice** – Extracts code that **actually** affects a variable during a specific program execution.
 - **Backward Slice** – Identifies statements that influence a particular point (e.g., a variable's value at a given statement).
 - **Forward Slice** – Identifies statements that are affected by a particular statement or variable.

Program Slicing Tools

1. CodeSurfer

- Provides **static program slicing** and **impact analysis**.
- Supports **C** and **C++** programs.
- Helps in **visualizing dependencies** between different parts of the code.

2. Atlas (by IBM Research)

- A **software analysis and visualization** tool that supports **program slicing**.
- Helps in **understanding complex software** by extracting dependencies.
- Supports multiple programming languages.

3. Frama-C

- A **formal analysis tool** for **C programs** that includes slicing capabilities.
- Helps in **extracting relevant portions of code** for security and correctness verification.
- Used in **critical software** domains like aviation and healthcare.

4. LLVM-Based Slicing Tools

- Various slicing tools are built on **LLVM infrastructure**, such as **LLVM-Slicer**, which supports **static analysis**.
- Used in **compiler optimizations** and **security analysis**.

5. WALA (T.J. Watson Libraries for Analysis)

- A slicing and program analysis framework for **Java** and **JavaScript**.
- Used for **static analysis**, **security analysis**, and **optimization**.

Thank You